Human Verification: `Containers.Rungs`

DAG and Graph Rungs of the Trust Ladder

Machine-generated from `lean/Rungs.lean` for review by Neil Ghani

July 2026

Abstract

This document presents every definition and theorem *statement* from the Lean 4 module `Containers.Rungs` in three parallel forms: the original Lean, a readable Haskell rendering, and precise mathematical English. Proofs are omitted — they are machine-checked by Lean. The purpose is to let a human verifier confirm that the formalisation captures the intended mathematics.

The file verifies two negative-result rungs of the directed-container trust ladder (§7 and §8 of the paper): the DAG rung shows that the Sawasdee Cellars wine graph extended with a Singapore transshipment is *not thin* (two distinct routes between the same nodes exist), and the Graph rung shows that a 2-node scrap loop produces an *infinite* position set.

Contents

1 Overview **1**

2 DAG rung: Sawasdee + Singapore transshipment (§7) **2**
- 2.1 Inductive type: `SawasdeeDAG.Facility` . . . 2
- 2.2 Inductive type: `SawasdeeDAG.Leg` . . . 2
- 2.3 Definition: `SawasdeeDAG.G'` . . . 2
- 2.4 Definitions: `directPath` and `transshipPath` . . . 3
- 2.5 Theorem: DAG rung (§7) . . . 3

3 Graph rung: 2-node scrap cycle (§8) **4**
- 3.1 Types: `ScrapLoop.Node`, `ScrapLoop.Edge`, `G_scrap` . . . 4
- 3.2 Definitions: `oneLap` and `laps` . . . 4
- 3.3 Structure: `Infinite` . . . 5
- 3.4 Theorem: `infinite_pos_rollingMill` . . . 5
- 3.5 Definition and theorem: `nontrivialEndo` and `nontrivialEndo_ne_root` . . . 5

4 Translation conventions **6**

1 Overview

A directed container whose position sets are all singletons is called *thin*: there is at most one route between any two nodes. The ladder in the paper has four rungs:

1. **Tree** — positions are singletons (trivially thin).
2. **Polytree / DAG without multiple routes** — still thin.
3. **DAG with multiple routes** — not thin (¬ Subsingleton).

4. **General graph with cycles** — position sets are infinite.

This file addresses rungs 3 and 4 with concrete witnesses.

2 DAG rung: Sawasdee + Singapore transshipment (§7)

2.1 Inductive type: `SawasdeeDAG.Facility`

Definition 2.1 (Extended facility set)**.** The set of **facilities** in the extended Sawasdee Cellars network consists of the original eleven nodes plus Singapore: twelve facilities in total.

Lean.

```
inductive Facility
  | vineyard1 | vineyard2 | vineyard3
  | sawasdee | melbournePort | singapore | thaiPort
  | distributor1 | distributor2
  | retail1 | retail2 | retail3
  deriving DecidableEq, Repr
```

Haskell.

```
data Facility
  = Vineyard1 | Vineyard2 | Vineyard3
  | Sawasdee | MelbournePort | Singapore | ThaiPort
  | Distributor1 | Distributor2
  | Retail1 | Retail2 | Retail3
  deriving (Eq, Show)
```

2.2 Inductive type: `SawasdeeDAG.Leg`

Definition 2.2 (Extended leg set)**.** The **legs** of the extended network consist of the original ten shipping legs plus two Singapore transshipment legs: `sing1` (Melbourne Port → Singapore) and `sing2` (Singapore → Thai Port).

Lean.

```
inductive Leg
  | v1s | v2s | v3s
  | smp
  | sea         -- direct Melbourne Port → Thai Port
  | sing1       -- Melbourne Port → Singapore
  | sing2       -- Singapore → Thai Port
  | td1 | td2
  | d1r1 | d1r2
  | d2r3
  deriving DecidableEq, Repr
```

Haskell.

```
data Leg
  = V1s | V2s | V3s | Smp
  | Sea | Sing1 | Sing2
  | Td1 | Td2 | D1r1 | D1r2 | D2r3
  deriving (Eq, Show)
```

2.3 Definition: `SawasdeeDAG.G'`

Definition 2.3 (Extended Sawasdee graph)**.** G' is the digraph with vertices `Facility`, edges `Leg`, and source/target maps `leg_src` and `leg_tgt` defined by pattern matching. It extends the original Sawasdee Cellars graph with the Singapore transshipment leg.

Lean.

```
@[reducible] def G' : Digraph where
  V := Facility
  E := Leg
  src := leg_src
  tgt := leg_tgt
```

Haskell.

```
data Digraph v e = Digraph { src :: e → v, tgt :: e → v }

g' :: Digraph Facility Leg
g' = Digraph legSrc legTgt
```

2.4 Definitions: `directPath` and `transshipPath`

Definition 2.4 (Witness routes)**.** Two distinct directed paths from `melbournePort` to `thaiPort` in G':

- directPath: the single leg sea, of length 1.
- transshipPath: the two-leg route sing1 followed by sing2, of length 2.

Lean.

```
def directPath : Path G' melbournePort thaiPort :=
  .cons (G := G') sea (.nil (G := G') thaiPort)

def transshipPath : Path G' melbournePort thaiPort :=
  .cons (G := G') sing1 (.cons (G := G') sing2 (.nil (G := G')
     thaiPort))
```

Haskell.

```
-- Paths represented as lists of legs (types approximate only)
directPath :: [Leg]
directPath = [Sea]

transshipPath :: [Leg]
transshipPath = [Sing1, Sing2]
```

2.5 Theorem: DAG rung (§7)

Theorem 2.5 (DAG rung — not thin)**.** The type of paths from `melbournePort` to `thaiPort` in G' is not a subsingleton: $\neg\,\mathrm{Subsingleton}(\mathrm{Path}_{G'}(\text{Melbourne Port}, \text{Thai Port}))$.

Proof witness: `directPath` has length 1 while `transshipPath` has length 2, so they are distinct. Hence the network $\mathcal{N}(G')$ is not thin at Melbourne Port.

Lean.

```
example : Not (Subsingleton (Path G' melbournePort thaiPort)) :=
  fun h ⇒ ...  -- length argument; proof omitted
```

Haskell.

```
-- The type of paths is not a singleton:
-- length directPath /= length transshipPath
dagRung :: Bool
dagRung = length directPath /= length transshipPath  -- True
```

3 Graph rung: 2-node scrap cycle (§8)

3.1 Types: `ScrapLoop.Node`, `ScrapLoop.Edge`, `G_scrap`

Definition 3.1 (Scrap-loop graph)**.** The scrap-loop graph G_{scrap} has:

- two nodes: `rollingMill` and `meltShop`;
- two directed edges: `rm2ms` (rolling mill → melt shop) and `ms2rm` (melt shop → rolling mill).

This is a 2-cycle: billets flow from the rolling mill, and scrap returns from the melt shop.

Lean.

```
inductive Node | rollingMill | meltShop
  deriving DecidableEq, Repr

inductive Edge | rm2ms | ms2rm
  deriving DecidableEq, Repr

@[reducible] def G_scrap : Digraph where
  V := Node
  E := Edge
  src := edge_src
  tgt := edge_tgt
```

Haskell.

```
data Node = RollingMill | MeltShop deriving (Eq, Show)
data Edge = Rm2ms | Ms2rm          deriving (Eq, Show)

gScrap :: Digraph Node Edge
gScrap = Digraph edgeSrc edgeTgt
```

3.2 Definitions: `oneLap` and `laps`

Definition 3.2 (Lap paths)**.** A single lap around the 2-cycle:

$$\text{oneLap} : \text{Path}_{G_{\text{scrap}}}(\text{rollingMill}, \text{rollingMill}),$$

consisting of the two edges `rm2ms` then `ms2rm`. The family $\texttt{laps} : \mathbb{N} \to \text{Path}_{G_{\text{scrap}}}(\text{rollingMill}, \text{rollingMill})$ iterates this lap: $\text{laps}(n)$ is the path that traverses the cycle exactly n times.

Lean.

```
def oneLap : Path G_scrap rollingMill rollingMill :=
  .cons (G := G_scrap) rm2ms
    (.cons (G := G_scrap) ms2rm (.nil (G := G_scrap) rollingMill))

def laps : Nat → Path G_scrap rollingMill rollingMill
  | .zero   ⇒ .nil (G := G_scrap) rollingMill
  | .succ n ⇒ (laps n).append oneLap
```

Haskell.

```
oneLap :: [Edge]
oneLap = [Rm2ms, Ms2rm]

laps :: Int → [Edge]
laps 0 = []
laps n = laps (n-1) ++ oneLap
```

3.3 Structure: `Infinite`

Definition 3.3 (Infinite type (bundled))**.** A type α is **infinite** if there exists an injective function $\mathbb{N} \to \alpha$:

$$\text{Infinite}(\alpha) \;\equiv\; \exists f : \mathbb{N} \to \alpha,\ \forall m\, n,\ f(m) = f(n) \Rightarrow m = n.$$

Lean.

```
structure Infinite (alpha : Type) : Prop where
  witness : ∀ (f : Nat → alpha), ∀ m n, f m = f n → m = n
```

Fidelity note. The Lean definition quantifies existentially (the witness field records the injection); the English and Haskell readings make this explicit.

Haskell.

```
-- A type a is infinite if there is an injection Nat → a.
-- In Haskell this is the class Infinite from Data.Infinite, or simply
   :
isInfinite' :: (Int → a) → (a → a → Bool) → (Int → Int → Bool)
   → Bool
isInfinite' f eq implies = True -- just a witness, not a proof
```

3.4 Theorem: `infinite_pos_rollingMill`

Theorem 3.4 (Graph rung — infinite position set)**.** The Σ-type $\sum_{w:\text{Node}} \text{Path}_{G_{\text{scrap}}}(\text{rollingMill}, w)$ is infinite. The injection is $n \mapsto (\text{rollingMill}, \text{laps}(n))$, which is injective because $\text{laps}(n)$ has length $2n$, a strictly increasing function of n.

Hence the position set of the directed container $\mathcal{N}(G_{\text{scrap}})$ at `rollingMill` is infinite, placing G_{scrap} on the top rung of the ladder.

Lean.

```
theorem infinite_pos_rollingMill :
    Infinite ((w : Node) * Path G_scrap rollingMill w) where
  witness := ∀ (n ⇒ ⟨rollingMill, laps n⟩), ...
```

Haskell.

```
-- The injection: n |→ (RollingMill, laps n)
-- Injectivity: length (laps m) = 2*m, so m /= n ⇒ laps m /= laps n
injection :: Int → (Node, [Edge])
injection n = (RollingMill, laps n)
```

3.5 Definition and theorem: `nontrivialEndo` and `nontrivialEndo_ne_root`

Definition 3.5 (Nontrivial endomorphism)**.** The **nontrivial endomorphism** at `rollingMill` is the one-lap position: the pair (rollingMill, oneLap) as an element of $\mathcal{N}(G_{\text{scrap}}).\text{Pos}(\text{rollingMill})$.

Theorem 3.6 (Endomorphism is non-trivial)**.** The nontrivial endomorphism is distinct from the root (the zero-lap empty path):

$$\text{nontrivialEndo} \;\neq\; \mathcal{N}(G_{\text{scrap}}).\text{root}(\text{rollingMill}).$$

Lean.

```
def nontrivialEndo : (Digraph.network G_scrap).Pos rollingMill :=
  ⟨rollingMill, oneLap⟩

theorem nontrivialEndo_ne_root :
    nontrivialEndo ≠ (Digraph.network G_scrap).root rollingMill
```

Haskell.

```haskell
nontrivialEndo :: (Node, [Edge])
nontrivialEndo = (RollingMill, oneLap)  -- length 2

root :: (Node, [Edge])
root = (RollingMill, [])  -- length 0

-- nontrivialEndo /= root  (different lengths)
```

4 Translation conventions

1. **Paths as lists.** In Lean, `Path G v w` is an indexed inductive type whose constructors carry type-level evidence of endpoint compatibility. In Haskell, paths are approximated as lists of edges `[Edge]`; the endpoint type indices are lost.
2. **Subsingleton.** Lean's `Subsingleton` is a typeclass saying a type has at most one element. The negation used in the DAG rung says: there exist two provably distinct elements.
3. **Injectivity witness.** Lean's bundled `Infinite` packs an injection together with its injectivity proof. The Haskell rendering only shows the injection function; the proof obligation is dropped.
4. **Proof bodies omitted.** Only definitions and theorem *statements* are translated. Lean checks the proofs; the human verifier checks that the *statements* match the intended mathematics.